· AI-Powered Research & Recommendation Agent

Type a company name · get a structured **intelligence report** plus a CEO-facing **AI-opportunity pitch**. Built for the AI/ML intern assessment.

The agent researches a company (live web search when keys are present, otherwise the model's own knowledge), then reasons about its challenges and where AI can help · grounded in that company's actual business, not generic boilerplate.

What it produces

For any company (e.g. *Sobha*, *Prestige Group*, *Brigade Group*):

1. **Company Overview** · what it does, industry, scale, geographic presence
2. **Key Business Information** · offerings, recent developments, expansion plans
3. **Potential Business Challenges** · tagged *operational / sales / customer-experience*, each **with reasoning**
4. **AI Opportunities** · company-specific, tagged by function (automation, customer-engagement, sales, operations, analytics, document-processing), each with a concrete business impact
5. **Personalized Pitch** · one page to the CEO: *why I reached out · what I found · what AI I'd build*

Plus: **PDF / Markdown export**, **report caching**, and **multi-company side-by-side comparison** with a relative AI-readiness matrix.

Quick start

```
pip install -r requirements.txt
cp .env.example .env          # then add ONE LLM key (or run Ollama
locally)
streamlit run app.py
```

CLI / smoke test (no UI):

```
python -m agent.report "Sobha"
python -m agent.report "Prestige Group" --provider groq --force
```

Getting a key in ~5 minutes (all free)

| Provider | Free? | Get a key | |---|---|---| | **Groq** (Llama 3.3 70B) | · free | console.groq.com | | **Google Gemini** (2.0 Flash) | · free tier | aistudio.google.com | | **OpenRouter** (free open models) | · free models | openrouter.ai | | **Ollama** (local) | · no key | `ollama serve` + `ollama pull llama3.2` | | Anthropic Claude | · paid | console.anthropic.com | | OpenAI GPT-4o | · paid | platform.openai.com |

Search (optional, enables **live web research**): Exa · Tavily · Firecrawl. With no search key the app runs in **LLM-knowledge mode** (a banner makes this explicit).

Architecture

```
flowchart TD
    User["User Input<br/>Company Name"] --> UI["Streamlit
UI<br/>app.py"]
```

```
    UI --> ReportBuild["report.build(company)<br/>agent/report.py"]
    ReportBuild --> CacheCheck{"Cache hit?<br/>agent/cache.py"}
    CacheCheck -->|"Yes"| Render["Render Report<br/>+ Export
(MD/PDF)"]
    CacheCheck -->|"No / Force"|
Research["research.gather()<br/>agent/research.py"]
    Research --> SearchCheck{"Search key<br/>configured?"}
    SearchCheck -->|"No"| LLMKnowledge["LLM-knowledge mode"]
    SearchCheck -->|"Yes"| SearchProviders["Live web search<br/>Exa
/ Tavily / Firecrawl"]
    LLMKnowledge --> Prompt
    SearchProviders --> Prompt["Build prompt<br/>+ embed sources"]
    Prompt --> LLMChain["LLM call<br/>agent/llm.py"]
    LLMChain --> ProviderFallback{"Fallback chain<br/>auto-detect
available"}
    ProviderFallback -->|"Groq (free)"| Groq
    ProviderFallback -->|"Gemini (free)"| Gemini
    ProviderFallback -->|"OpenRouter"| OpenRouter
    ProviderFallback -->|"Ollama (local)"| Ollama
    ProviderFallback -->|"Claude (paid)"| Claude
    ProviderFallback -->|"GPT-4o (paid)"| GPT4o
    Groq --> StructuredJSON["Structured JSON<br/>+ Defensive
parser"]
    Gemini --> StructuredJSON
    OpenRouter --> StructuredJSON
    Ollama --> StructuredJSON
    Claude --> StructuredJSON
    GPT4o --> StructuredJSON
    StructuredJSON --> Cache[("Write to cache<br/>cache/*.json")]
    Cache --> Report["Report dataclass<br/>5 sections"]
    Report --> Render
    Render --> MD["·· Markdown<br/>agent/export.py"]
    Render --> PDF["·· PDF<br/>agent/export.py"]
    Render -->
Compare["Multi-company<br/>Comparison<br/>agent/compare.py"]
    style User fill:#e1f5fe,stroke:#0288d1
    style UI fill:#bbdefb,stroke:#1976d2
    style ReportBuild fill:#fff3e0,stroke:#f57c00
    style Groq fill:#e8f5e9,stroke:#388e3c
    style Gemini fill:#e8f5e9,stroke:#388e3c
    style OpenRouter fill:#e8f5e9,stroke:#388e3c
    style Ollama fill:#e8f5e9,stroke:#388e3c
    style Claude fill:#fce4ec,stroke:#c62828
    style GPT4o fill:#fce4ec,stroke:#c62828
    style LLMKnowledge fill:#fff9c4,stroke:#fbc02d
```

Why these choices

- **Single structured JSON call** instead of 5 per-section calls · ~5× fewer tokens, coherent output, fewer failure points (matters on free tiers).
- **Provider abstraction with lazy imports** · a missing SDK for an unused provider never breaks startup; swap models with one dropdown / env var.
- **Hybrid everything** · works whether you have a paid Claude key, a free Groq key, or just Ollama on

your laptop; live web search is additive, not required.
- **The prompt is the product.** It forces specificity: challenges must explain *why they apply to this company*; AI opportunities must reference the company's real offerings and name a concrete win · generic answers are explicitly disallowed.

Module map

File	Responsibility
`agent/config.py`	Detect available providers from env; priority + fallback order
`agent/llm.py`	6 LLM adapters behind one `generate()`; fallback chain
`agent/research.py`	3 search adapters · normalized `Source` records
`agent/report.py`	Prompt + orchestration + defensive JSON parsing
`agent/cache.py`	Disk cache keyed by company + model
`agent/export.py`	Markdown + PDF (fpdf2, pure-python)
`agent/compare.py`	Multi-company reports + comparison matrix
`agent/landscape.py`	Competitive landscape (3-5 companies: positioning, gaps, recommendations)
`app.py`	Streamlit UI (single + compare + landscape tabs)
`tests/`	23 unit tests (config, parsing, cache, export, fake-LLM build, landscape)

AI tools used

- **LLMs (pluggable):** Groq (Llama 3.3 70B), Google Gemini 2.0 Flash, OpenRouter free models, local Ollama, Anthropic Claude, OpenAI GPT-4o.
- **Web research:** Exa (neural search + contents), Tavily (LLM-tuned search), Firecrawl (search + scrape).
- **Built with:** Claude Code (Anthropic) for design, implementation, and tests.

Challenges faced & how they were solved

Challenge	Solution
Free/open models return **inconsistent JSON** (code fences, prose around it)	Defensive parser strips fences + extracts the outer `{…}`one automatic "return valid JSON only" repair retry
Should run **with or without** paid keys / internet	Auto-detect providers; graceful fallback to LLM-knowledge mode with a visible banner
One provider being down shouldn't break a demo	`generate()` walks a **fallback chain** across all available LLMs
Free-tier **context limits**	Source text clipped per-source and capped to a token budget before prompting
PDF on Windows without system libraries	`fpdf2` (pure-python); fixed cursor reset so `multi_cell` keeps full page width
Avoid **generic AI advice** (the actual grading risk)	Prompt forces per-company reasoning + references to real offerings; categories/functions enforced in schema

---

Testing

```
pip install pytest
python -m pytest -q          # 23 passed
```

Covers provider detection & priority, JSON extraction (fenced / prose-wrapped), category normalization, cache round-trip, markdown/PDF shape, and a full `report.build()` with a fake LLM (including the bad-JSON repair path).

Project layout

```
INTERNSHIP ASSESSMENT/
```

```
··· app.py                # Streamlit UI
··· agent/                # config, llm, research, report, cache,
export, compare
··· tests/                # pytest suite
··· cache/                # generated reports (gitignored)
··· requirements.txt
··· .env.example
··· docs/superpowers/specs/2026-06-17-...-design.md
```